

Feature Selection for RCT Data

Regression Pipeline Documentation

1. Overview

This document describes the feature selection methods used for RCT (Randomized Controlled Trial) data in the regression pipeline. The pipeline handles high-dimensional clinical data where features outnumber samples.

Two feature selection methods are available: **GRACES** (graph-based) and **DeepFS** (autoencoder-based).

2. GRACES

2.1 Method

GRACES uses a GraphSAGE neural network to learn feature importance. Samples are connected via cosine similarity edges. After training, gradient-based saliency identifies the most predictive features.

2.2 Steps

- Standardize features (zero mean, unit variance)
- Build sample graph using cosine similarity (top 5% edges)
- Train GraphSAGE on regression task (MSE loss)
- Compute saliency: mean absolute gradient per feature
- Select top-k features by saliency score

2.3 Implementation

```
# Graph construction
sim = |cosine_similarity(samples)|
edges = sim >= quantile(sim, 0.95)

# Training
model = GraphSAGE([64, 32] -> 1)
loss = MSELoss(predicted, target)

# Feature selection
saliency = mean(|grad(loss, input)|, dim=0)
selected = topk(saliency, n_features)
```

2.4 Parameters

```
n_features    = 50
hidden_size   = [64, 32]
epochs        = 100
alpha         = 0.95
lr            = 0.001
batch_size    = 8
```

2.5 Reference

Chen et al. arXiv:2211.14144

3. DeepFS

3.1 Method

DeepFS trains a supervised autoencoder, then ranks features by their Rank Distance Correlation (RdCorr) with the learned latent embedding. Captures non-linear relationships.

3.2 Steps

- Standardize features
- Train supervised autoencoder (MSE + reconstruction loss)
- Extract latent embeddings
- Compute RdCorr between each feature and embedding
- Select top-k features by RdCorr

3.3 Implementation

```
# Architecture
Encoder: input -> 100 -> ReLU -> latent_dim
Decoder: latent_dim -> 100 -> ReLU -> input
Head: latent_dim -> 1 (regression)

# Loss
loss = MSE(y_pred, y) + 0.01 * MSE(x_recon, x)

# RdCorr computation
For each feature i:
    score[i] = RdCorr(feature_i, latent_embedding)
```

3.4 Parameters

```
n_features    = 50
latent_dim    = 15
epochs        = 500
lr            = 0.001
lambda_reg    = 0.01
```

3.5 Reference

Li et al. arXiv:2204.01682

4. RCT Data Preprocessing

4.1 Standardization

All features are standardized before selection:

```
X_scaled = (X - mean) / std
```

4.2 Data Reduction

Optional removal of samples to simulate low-sample scenarios:

```
n_keep = n_original * (1 - reduction_pct)
indices = random_choice(n_original, n_keep)
```

4.3 SMOGN (RCT-specific)

Handles imbalanced target distributions in RCT data. Generates synthetic samples for rare outcome values while preventing cross-tail pollution.

```
# Define rare regions
low_rare: y <= 5th percentile
high_rare: y >= 95th percentile

# Generation rules
- Interpolate only within same tail
- Cap synthetic samples at 100% of rare count
- Preserves outcome distribution integrity
```

5. Usage

5.1 CLI Commands

```
# GRACES with SMOGN
python main.py --data rct_data.csv --target outcome \
  --method GRACES --features 50 --smogn

# DeepFS without SMOGN
python main.py --data rct_data.csv --target outcome \
  --method DEEPFS --features 50
```

5.2 Available Methods

```
GRACES      - GRACES feature selection + TabPFN
DEEPFS      - DeepFS feature selection + TabPFN
GRACES_TABPFN - Same as GRACES
DEEPFS_TABPFN - Same as DEEPFS
XGBOOST     - XGBoost without feature selection
TABPFN      - TabPFN without feature selection
```

5.3 Pipeline Order

1. Load RCT data
2. Train/test split (80/20)
3. Data reduction (optional)
4. SMOGN augmentation (optional)
5. Feature selection (GRACES or DeepFS)
6. Model training on selected features
7. Evaluation (RMSE, MAE, R2)

6. Output

The selector returns indices of selected features:

```
selector.fit(X_train, y_train)
X_selected = selector.transform(X)

# Attributes
selector.selected_indices_ # Feature indices
selector.feature_scores_  # Scores (DeepFS only)
```